

Getting DOOM to Run on an STM32U585

A debugging field report

Porting NXP's Doom-MCX (prBoom) to Zephyr on the Arduino UNO Q: the constraints, the crashes, and the tools that found them — native_sim + Valgrind for the fast lane, OpenOCD/SWD and GDB for the metal.

Target STM32U585 (Cortex-M33 @160 MHz, 768 KB SRAM, 2 MB flash)

Display 13×8 charlieplex LED matrix (104 px, 8 grey levels)

Stack Zephyr 4.4.x, picolibc, arm-zephyr-eabi (Zephyr SDK 0.16)

Audience embedded engineers who like the gory details

1. The target, and why it is tight

DOOM (1993) was written for a 486 with 4 MB of RAM and a hard disk holding a ~4 MB WAD. The job here was to make it run on a microcontroller: the STM32U585 on an Arduino UNO Q, rendering into a 13×8 LED matrix and taking input over the board's SPI bridge. The U585 is a generous MCU, but it is still an MCU, and three of its numbers define the entire effort:

Resource	STM32U585	Consequence for DOOM
CPU	Cortex-M33 @ 160 MHz, FPU, MPU, TrustZone (TZEN=0)	Fast enough for software 3D at a few fps; FPU unused by the fixed-point engine but handy for the display path.
SRAM	768 KB, 0x20000000 - 0x200C0000	Comfortable for the zone heap + framebuffers once they are sized sanely.
Flash	2 MB internal, dual-bank, NO external flash	The hard wall. The game code AND the WAD must both live in 2 MB.

There is no external QSPI/SDRAM on this board, so every classic 'put the WAD on a memory-mapped flash' trick is off the table. That single fact drove the port's first major decision and its first class of bugs.

2. Choosing a base port

Two candidates. **floppes/stm32doom** is a beautiful Chocolate-Doom port, but it assumes ~8 MB of external SDRAM and a USB-stick WAD — neither exists on the UNO Q. **NXP Doom-MCX** is a prBoom derivative already proven on the FRDM-MCXN947 (also a Cortex-M33, 512 KB SRAM), structured as a Zephyr application with the SDL hooks replaced by four platform functions (`I_FinishUpdate` / `I_SetPalette` / `I_ProcessKeyEvents` / `I_InitGraphics`). It bakes the WAD into flash as a C array and converts the indexed frame to the panel format line-by-line with no second framebuffer. That is the right shape for an MCU, so Doom-MCX (GPLv2, prBoom lineage) was the base.

Why the base choice is also a debugging decision

Doom-MCX targets Zephyr 4.1.0 and is normally exercised on `native_sim` with `glibc`. We built it on Zephyr 4.4.x with `arm_picolibc`. A port across both a **three-minor-version Zephyr gap** and a **libc swap** means the first bugs are environmental, not logical — and they must be cleared before any DOOM-specific bug is even visible. Establishing a known-good baseline (does it build and boot the title screen at all?) was step zero.

3. The flash gate: 2 MB versus a 4 MB game

The full DOOM1.WAD baked as a C array is roughly 3.84 MB of binary — it does not fit, full stop. The link failed by ~1.98 MB, which also gave a clean measurement: the engine itself is ~228 KB, leaving a WAD budget of about 1.82 MB. The fix was **Squashware** (fraggle/squashware), a shrunken but complete shareware WAD. The v1.3 full edition is 1,703,241 bytes — it fits, with all nine E1Mx maps intact.

- **Bake the WAD:** convert the 1.62 MB WAD to a C array (`const unsigned char doom_iwad[1703241]`) and select it via `#include "iwad/squashware.c"` in `doom_iwad.c`.
- **Drop MCUboot for the DOOM image:** an A/B layout would halve the usable flash. DOOM ships as a single image flashed straight to 0x08000000. (The OTA demo on this board is a separate mode — `mcuboot` + a signed app in `slot0` — that you switch to by reflashing.)

Final footprint: **FLASH 1,948,000 B (92.9%)**, **RAM 50.6%**. The flash headroom is thin but real; RAM is comfortable once the framebuffers are sized correctly (see §10).

4. Toolchain landmines: Zephyr 4.4 + picolibc

Three environmental failures stood between ‘clones’ and ‘boots’. None are DOOM bugs; all are the kind of thing that eats an afternoon if you do not recognise the signature.

4.1 A symbol named gamma

picolibc declares `gamma()` (the log-gamma function). prBoom’s menu code has an enum member literally named `gamma`. The collision is a build break in `m_menu.c`. Fix: rename the enumerator to `gamma_item` (declaration and use).

4.2 POSIX AEP drags in a struct picolibc does not have

Enabling the full `CONFIG_POSIX_API` pulls in the realtime AEP, including `posix/options/timer.c`, which references `sigev_notify_function / sigev_notify_attributes` — fields picolibc’s struct `sigevent` lacks on Zephyr 4.4. DOOM needs only a handful of libc calls, so the fix is to enable just the libc extension and the two subsystems it uses, and leave timers/signals off:

```
# DOOM needs free/printf/stat/write, NOT the realtime AEP.
CONFIG_POSIX_C_LIB_EXT=y
CONFIG_POSIX_DEVICE_IO=y
CONFIG_POSIX_FILE_SYSTEM=y
# CONFIG_POSIX_API is not set
# CONFIG_POSIX_TIMERS is not set
# CONFIG_POSIX_SIGNALS is not set
```

4.3 gettimeofday() does not exist

The engine’s tic clock, `I_GetTime()`, calls `gettimeofday()`; picolibc without the AEP does not provide it. A four-line shim backed by the kernel uptime is enough:

```
int gettimeofday(struct timeval *tv, void *tz) {
    int64_t ms = k_uptime_get();
    tv->tv_sec = ms / 1000;
    tv->tv_usec = (ms % 1000) * 1000;
    return 0;
}
```

Keep this shim honest

`I_GetTime()` converts to tics as `tv_sec*TICRATE + tv_usec*TICRATE/1e6`. If your `gettimeofday()` returns a constant, DOOM never advances a tic, the main loop never calls `I_FinishUpdate`, and the render thread blocks forever after frame 1. A frozen screen with a live CPU often means a dead clock, not a dead renderer — verify the tic actually increments before you suspect anything else.

5. Two debug surfaces: native_sim versus the metal

The single most useful decision in this port was to keep **two** build targets alive at all times and to know which one to reach for:

	native_sim (x86)	arduino_uno_q (STM32U585)
Cycle time	seconds: <code>cmake + ninja + run</code>	~2 min SWD flash per change
Memory tooling	Valgrind, ASan, gdb, core dumps	SWD peeks, fault registers only
Faithful to HW?	No: x86, SDL display, 4 MB heap	Yes: the real panel + timing

	native_sim (x86)	arduino_uno_q (STM32U585)
Best for	logic bugs: map loader, struct casts, OOB	HW bugs: display, SPI, alignment, timing

The discipline: **reproduce on native_sim first**. If a bug shows up there, you get Valgrind and a sub-ten-second loop. Only bugs that genuinely depend on the silicon (the charlieplex path, SPI slave timing, unaligned access) earn a flash cycle. Conversely, do not over-trust native_sim: it uses an SDL display at 640×360, so it cannot validate the 13×8 downscale — a frame that looked ‘stuck’ on native_sim turned out to be a sim-only artifact that ran fine on the matrix.

6. The real bug: a WAD the loader cannot read

With the environment clean, the game booted to the title screen, then crashed hard the moment a level loaded. On hardware it was a fault deep in texture/BSP setup; the breakthrough was reproducing it on native_sim in seconds:

```
P_GroupLines: Subsector a part of no sector!
P_GroupLines: Subsector a part of no sector!
... (repeated) ...
Segmentation fault (core dumped)      # exit 139
```

That message is diagnostic gold. Every subsector failing to find its sector means the seg / sidedef / sector linkage is garbage — which points straight at the map loader. The root cause: Doom-MCX’s loaders **direct-cast raw on-disk WAD lumps onto the engine’s in-memory structs**, instead of converting between the two layouts. That only works if the WAD has been rewritten ahead of time into the engine’s memory format.

6.1 On-disk is not in-memory

The on-disk map lumps are compact arrays of indices and 16-bit coordinates. The engine’s runtime structs are wider and hold *resolved* pointers and fixed-point values. Casting one onto the other reads adjacent fields as garbage:

Lump	On-disk record	In-memory record	Conversion
VERTEXES	mapvertex_t: int16 x, y (4 B)	vertex_t: fixed_t x, y (8 B)	$x = \text{SHORT}(x) \ll \text{FRACBITS}$
SEGS	mapseg_t (12 B): v1,v2,angle,linedef,side,offset (u16)	seg_t: resolved v1/v2, angle_t, fixed_t offset, side/line/sector nums	look up vertices; angle<<16; offset<<FRACBITS; sidenum = ldef->sidenum[side]
SIDEDEFS	30 B: int16 texoff,rowoff; char[8]x3 tex names; int16 sector	side_t: numeric texture ids + sector*	R_CheckTextureNumF orName(name); -1 -> 0

The smoking gun was in the original, working doom1.c shipped with Doom-MCX: its E1M1 **SEGS lump has size 0**, and the SSECTORS / NODES are resized. In other words, NXP runs a *build-time WAD preprocessor* that rewrites the BSP and vertex lumps into the engine’s in-memory layout. That tool is not in the repository. A stock WAD — Squashware or otherwise — uses the standard on-disk format and therefore crashes the cast-based loader.

6.2 The fix: convert on load (stock prBoom behaviour)

Rather than reverse-engineer NXP’s preprocessor, the loaders were rewritten to convert on load — allocate the in-memory array, then resolve each field — exactly as upstream prBoom does. Representative of the set (P_LoadVertexes / P_LoadLineDefs / P_LoadSegs / P_LoadSideDefs):

```

static void P_LoadSegs(int lump) {
    const mapseg_t *ml = W_CacheLumpNum(lump);
    int n = W_LumpLength(lump) / sizeof(mapseg_t);
    seg_t *segs = Z_Malloc(n * sizeof(seg_t), PU_LEVEL, 0);
    _g->segs = segs;
    for (int i = 0; i < n; i++) {
        const mapseg_t *m = &ml[i];
        seg_t *li = &segs[i];
        li->v1 = _g->vertexes[(unsigned short)SHORT(m->v1)];
        li->v2 = _g->vertexes[(unsigned short)SHORT(m->v2)];
        li->angle = (angle_t)SHORT(m->angle) <<< 16;
        li->offset = (fixed_t)SHORT(m->offset) <<< FRACBITS;
        int linenum = (unsigned short)SHORT(m->linedef);
        const line_t *ldef = &_g->lines[linenum];
        int side = SHORT(m->side);
        li->sidenum = ldef->sidenum[side];
        li->frontsectornum = (li->sidenum != NO_INDEX)
            ? _g->sides[li->sidenum].sector - _g->sectors : NO_INDEX;
        /* backsector via ldef->sidenum[side^1] when ML_TWOSIDED ... */
    }
}

```

After the rewrite the counts came out sane (verts=402, sectors=82, sides=273, lines=431, subs=197) with zero P_GroupLines warnings, on both native_sim and HW.

Two adjacent fixes were needed for the stripped Squashware WAD: the status bar and HUD draw routines were gated out for the matrix build (Squashware strips the STGANUM* digit graphics, so ST_Drawer dereferenced NULL patches), and the title screen was replaced with a direct G_DeferedInitNew() into E1M1.

7. The red herrings (and how each was ruled out)

Before the WAD-loader root cause was found, three other hypotheses looked plausible. Each was investigated, and although none was the cause, two were real latent bugs worth keeping. This is the honest part of any field report.

- **Stack overflow.** The renderer recurses (R_RenderBSPNode) and level setup uses large stack locals. The 8 KB main stack genuinely was too small, so it was raised to 32 KB and the MPU + HW stack protection were enabled. Real improvement — but the crash persisted, now faulting cleanly instead of corrupting RAM silently.
- **Unaligned access.** The WAD's infotableofs is odd (0x19c029), so packed multi-byte reads could LDRD-fault. Adding -mno-unaligned-access was correct hardening, but the crash address merely moved — symptom, not cause.
- **Display downscale stride.** The matrix downscale indexed the frame with CONFIG_DOOM_X_RES (the 565 width) instead of SCREENWIDTH = X_RES/2 (the engine packs two columns per output pixel). This overran the backbuffer — a genuine bug, fixed — but downstream of the load crash.

The lesson the red herrings teach

Every one of these 'wrong' theories produced a memory-safety improvement, and on bare metal that is not wasted work: tightening the MPU and the alignment behaviour **changed the crash from silent corruption into a precise fault with a valid BFAR** (next section). Sometimes you fix the instrumentation before you fix the bug.

8. Reading a bus fault on bare metal

On hardware there is no Valgrind. The Cortex-M33 fault status registers are the equivalent, and they are precise if you read them correctly. The fault here was caught with a hardware breakpoint on the fault path, then decoded from CFSR / BFAR:

```
# OpenOCD running on the device (linuxgpiod SWD bitbang):
> mrw 0xE000ED28      ; CFSR (Configurable Fault Status)
   0x00008200
> mrw 0xE000ED38      ; BFAR (Bus Fault Address)
   0x200C2048
```

Decoding CFSR = 0x00008200:

Field	Bits	Value	Meaning
UFSR	[31:16]	0x0000	no usage fault (so NOT div-by-zero / undef instr)
BFSR	[15:8]	0x82	BFARVALID (0x80) + PRECISERR (0x02): precise data bus error, BFAR valid
MMFSR	[7:0]	0x00	no MPU region fault

So it is a **precise data bus fault** and BFAR is trustworthy. BFAR = 0x200C2048. SRAM ends at 0x200C0000 (0x20000000 + 768 KB), so the faulting access is 0x2048 bytes **past the top of RAM** — a wild pointer read, consistent with the cast-based loader walking off a mis-sized array. The fault register decode and the native_sim Valgrind report pointed at the same code from two completely different directions, which is how you know you have the root cause and not a coincidence.

Catching the fault, not its aftermath

A naive halt lands you in `z_arm_fatal_error` with the original context already unwound. Set a hardware breakpoint on the fault entry (bp `z_arm_usage_fault` / the BusFault vector) so you stop with the **exception stack frame** intact, then read the stacked PC / LR from the ESF to get the faulting instruction. CFSR/BFAR are latched, so they survive to the handler — but the call stack does not.

8.1 GDB for layout, not just for stepping

You rarely single-step a 35-fps game over SWD. GDB's more valuable role here was static: extracting struct offsets from the ELF so raw memory reads could be interpreted. There is no need to guess where a field lives — ask the DWARF:

```
arm-zephyr-eabi-gdb -q -batch build/zephyr/zephyr.elf \
-ex 'printf "gametic=%d\n", (int)&((globals_t*)0)-&gametic' \
-ex 'printf "player=%d\n", (int)&((globals_t*)0)-&player' \
-ex 'printf "mo=%d\n", (int)&((player_t*)0)-&mo' \
-ex 'printf "angle=%d\n", (int)&((mobj_t*)0)-&angle'
# gametic=484 player=232 mo=0 angle=32
```

8.2 SWD memory peek: the printf you did not compile in

The most-used technique across the whole port was non-intrusive halt / read / resume over SWD. nm gives the address of the `_g` pointer; dereference it, add the GDB offsets, and you can watch live engine state without a single added printf or a reflash:

```
> halt
> mrw 0x20000830      ; _g (pointer to globals)
   0x20017DB4
> mdw 0x20017DB4+484 1 ; gametic
   0x000008F4      ; = 2292
> resume
... wait 2.5 s, halt again ...
> mdw 0x20017DB4+484 1
   0x0000097C      ; +136 in 2.5 s ~= 35 tics/s -& the loop is LIVE
```

This one trick answered the ‘frozen or running?’ question definitively. The matrix looked static, but gametic was advancing at exactly 35 tics/s and the main loop blocks on the display semaphore every frame — so the display thread *had* to be completing each frame. The image was static only because the player was standing still. To prove the render path end-to-end, we wrote the player’s view angle directly and watched the framebuffer change:

```
> mww 0x20033BB8 0xC0000000 ; player.mo->angle = 180 deg (BAM)
> resume ; the matrix image rotates 180 deg
# framebuffer at *cplx_fb went from a dark wall to a bright room.
```

9. native_sim + Valgrind: the fast lane

Because native_sim is an ordinary x86 ELF, the entire host toolbox applies. The level-load crash that took hardware breakpoints and CFSR decoding to corner on the MCU was, on native_sim, a one-line Valgrind run that named the offending function and the invalid read width directly. The workflow:

```
# Host build (NOT the docker builder: it lacks SDL2 and is non-root).
python3 -m venv /tmp/zvenv && . /tmp/zvenv/bin/activate
pip install -r zephyr/scripts/requirements-base.txt
export ZEPHYR_BASE=$PWD/zephyr ZEPHYR_TOOLCHAIN_VARIANT=host
cmake -GNinja -B build_ns -S doom-mcx -DBOARD=native_sim/native/64
ninja -C build_ns

# Reproduce + localise in one shot:
SDL_VIDEODRIVER=dummy valgrind --error-exitcode=1 \
    build_ns/zephyr/zephyr.exe
```

Why x86 repro is worth the dual-target maintenance

A 2-minute flash makes bisection painful: ten iterations is half an hour of watching a progress bar. The same ten iterations on native_sim is under two minutes total, with Valgrind annotating each one. Spend the up-front effort to keep the simulator build green — it pays for itself the first time you have a memory bug. Just remember its blind spots (display, timing, alignment) and finish on hardware.

10. Making 104 pixels legible

Getting DOOM to render was not the same as getting it to be *readable* on a 13×8 grid. Two problems, both fixed in the downscale path.

10.1 Aspect ratio

The engine renders at SCREENWIDTH = X_RES/2 by SCREENHEIGHT = Y_RES. With X_RES=400, Y_RES=320 that is a **200×320 portrait** frame — taller than wide — box-downscaled into a 13×8 **landscape** (1.625:1) matrix. The vertical axis was crushed ~40:1 against ~15:1 horizontal, smearing the scene into noise. Dropping Y_RES to 128 gives a landscape 200×128 frame (~1.56:1, close to 13:8) with the correct DOOM proportions — and frees ~75 KB of RAM. The width was deliberately left at 200: an earlier 160×128 attempt (SCREENWIDTH=80) tripped small-width assumptions in the column renderer.

10.2 Eight grey levels, used

DOOM scenes are dark; a naive linear map to 0..7 left the panel using only 0..2. The fix is a per-frame auto-exposure: keep full 0.255 luma per cell, then stretch this frame’s [min,max] across the range and apply a brightening gamma via a threshold table. Verified over SWD that the framebuffer now spans the full 0..7.

```

/* per cell: avg luma 0..255 -&gt; stretch -&gt; gamma threshold -&gt; 0..7 */
int range = mx - mn; if (range < 24) range = 24; /* don't amplify flat frames */
static const uint8_t thr[7] = {10,32,62,100,145,197,240}; /* gamma < 1 */
int s = ((avg[i] - mn) * 255) / range; if (s > 255) s = 255;
uint8_t lvl = 0; for (int k = 0; k < 7; k++) if (s >= thr[k]) lvl = k + 1;
cplx_fb[i] = lvl;

```

11. The input path: SPI slave, RDY, CRC, and an adb gotcha

Input reuses the board's SoC↔STM32 SPI link. The STM32 runs an **SPI3 slave** (2 MHz, mode 0, MSB, 8-bit) and the QCS Linux side is the /dev/spidev master. A 64-byte block protocol carries key state, with a hardware RDY line (PG13 == SoC gpiochip1:70) for flow control and a CRC-16/CCITT-FALSE per block:

```

[0] 0xA5 [1] ver [2] type [3] seq [4] len [5..] payload [62..63] crc16-le
TYPE_KEYS = 0x20, payload[0] = bitmask:
    bit0 fwd bit1 back bit2 left bit3 right
    bit4 fire bit5 use bit6 strafel bit7 strafR

```

The slave parks in `spi_transceive()`, raises RDY when armed, drops it while parsing. A poll on the engine's main thread edge-detects the bitmask and posts DOOM key events. Two robustness details earned their keep:

- **Watchdog.** If the host goes quiet (crash, unplug), the last bitmask would otherwise stick — the player runs into a wall forever. A 300 ms timeout clears all keys. Verified by SWD: after the host exits mid-'turn right', the angle stops changing within the window.
- **The transport bug.** The first design captured keys on the laptop and piped them to the device via `adb exec-out`. Nothing moved. SWD showed blocks arriving with zero CRC errors but a zero key mask: `adb exec-out` **does not forward stdin**. `adb shell -t` allocates a PTY and does, so the controller was moved onto the device and launched over an adb shell.

Two more gotchas from the host side

The stock device python is a **minimal build with no termios/tty module**, so raw mode is set with `stty -echo -icanon min 1 time 0` around the controller rather than from Python. And `adb shell <cmd>` runs **without a PTY** unless you pass `-t` — without it your local terminal stays cooked, keys echo locally and only flush on Enter.

12. Flashing without wedging the link

A 1.94 MB image over a bit-banged SWD link is slow, and two sharp edges cost real time before they were understood:

- **Do not verify over SWD.** OpenOCD's `verify_image` reads the whole 1.94 MB back at ~15 KiB/s — minutes on top of the write, and it hung outright for ~12 on one run. The write itself is reliable; drop the verify. A custom `flash-fast.sh` does write + reset only.
- **reset halt, not halt.** A bare `halt` on a busy target gives 'target was in unknown state when halt was requested' and the flash algorithm can time out. `reset halt` gives the flash loader a clean machine.
- **Never poll over adb during an SWD flash.** Concurrent adb access while OpenOCD drives SWD wedges the link and forces a physical reset. Start the flash in the background and leave the device alone until it finishes.

13. Playbook: what to carry to the next port

- **Two targets, always.** A host simulator for logic + Valgrind, the metal for everything hardware. Reproduce on the simulator first; finish on hardware.
- **Clear the environment before you trust a bug.** Across a libc or Zephyr version gap, the first failures are toolchain (symbol collisions, missing libc calls), not your code.
- **A frozen screen with a live CPU is usually a dead clock or a one-shot handshake,** not a dead renderer. Prove the tic advances first.
- **Decode the fault registers, do not guess.** CFSR tells you the class; BFAR/MMFAR the address; the ESF the instruction. Tighten the MPU so corruption becomes a precise fault.
- **SWD halt/read/resume is a free printf.** nm + GDB offsets + a pointer deref let you watch any global live, no reflash, no instrumentation.
- **Red herrings that harden memory safety are not wasted** — they often turn an undiagnosable corruption into a catchable fault.
- **Know your flash link's limits.** Skip the SWD read-back verify; use reset halt; never share the debug link with another tool mid-operation.

The result: DOOM running from internal flash on an STM32U585, E1M1 rendering to a 13×8 LED matrix at the engine's native tic rate, driven by WASD over the on-board SPI bridge — 92.9% of flash, 50.6% of RAM, and a fistful of fault registers later.